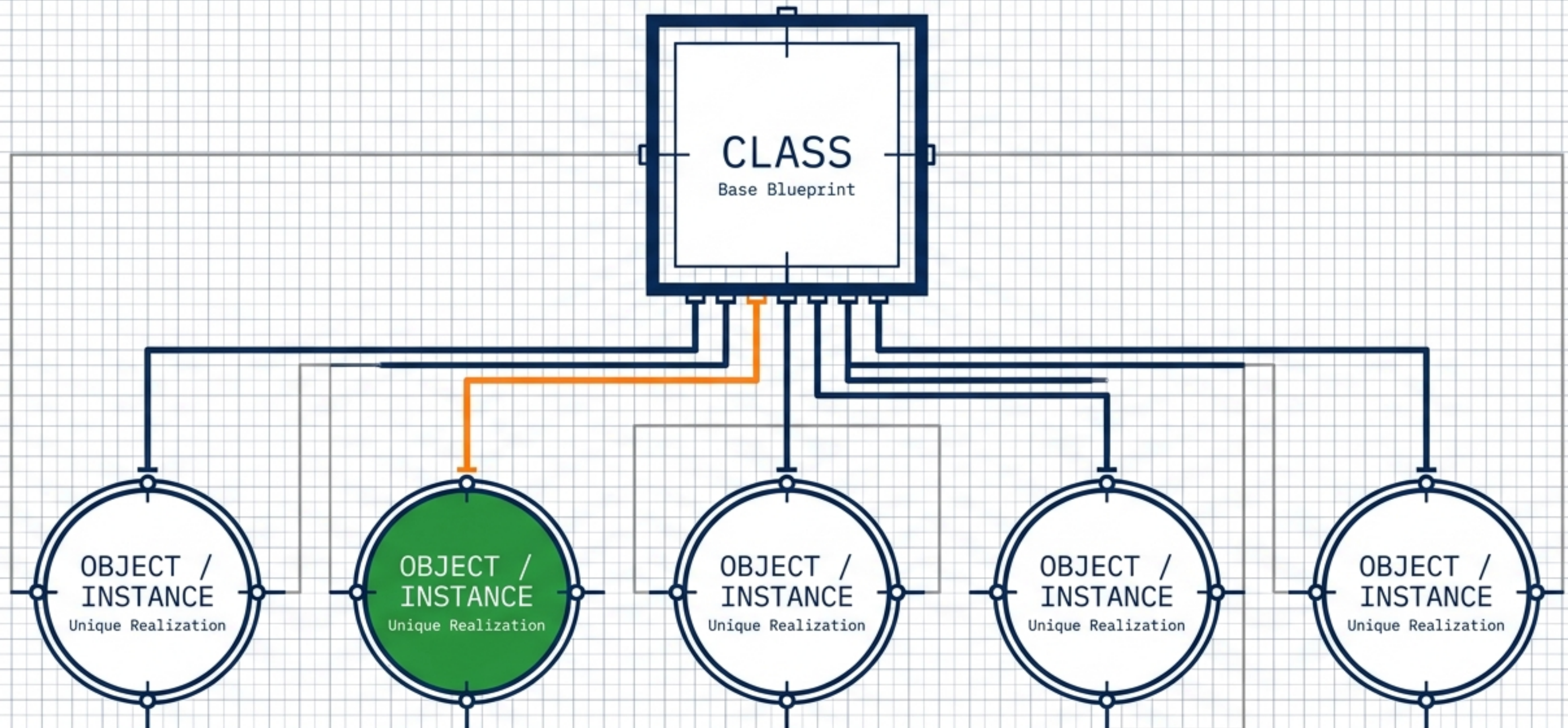


Introdução à Orientação a Objetos (POO) em Python

Do conceito abstrato à construção de sistemas passo a passo.



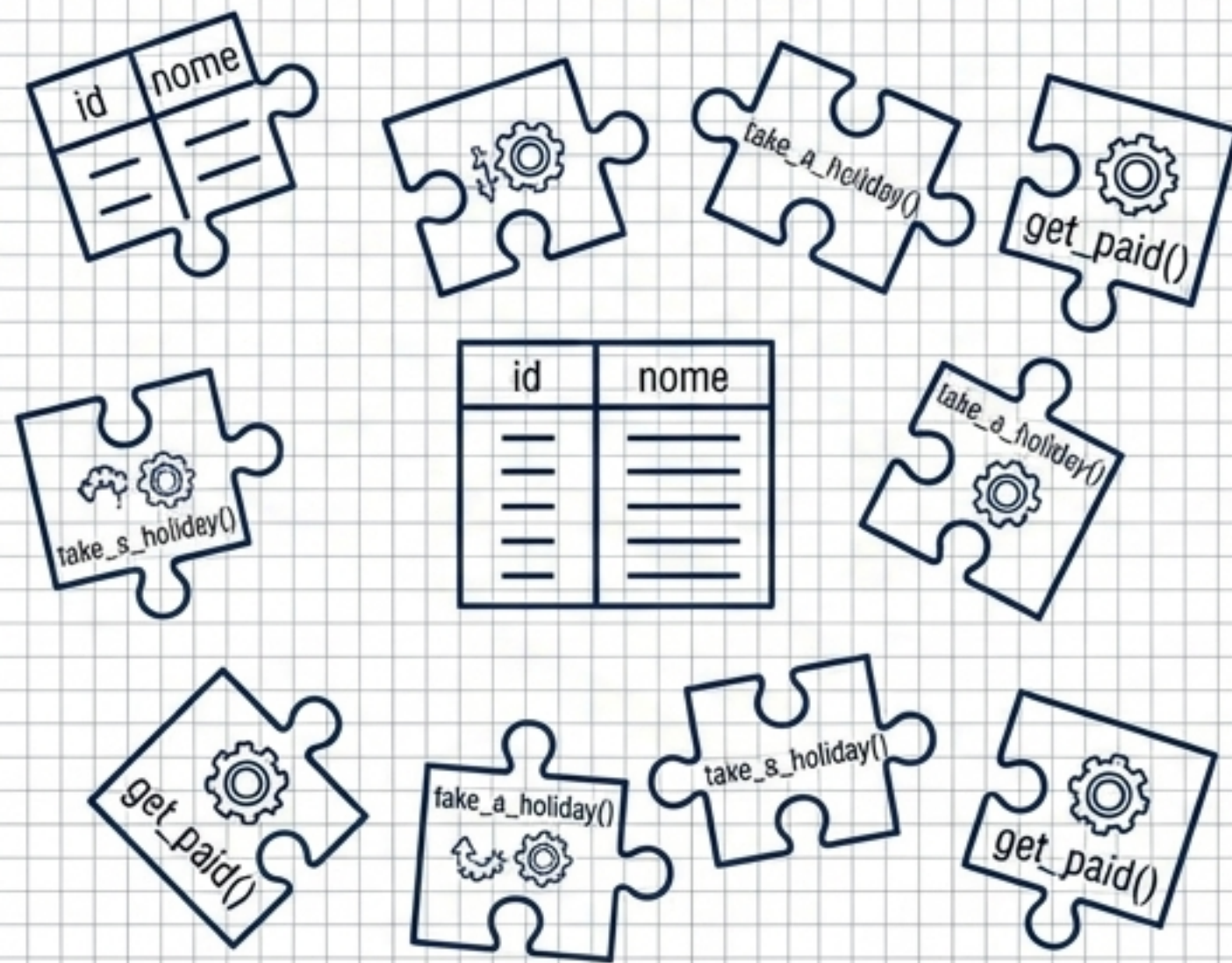
Roteiro de Montagem (Sumário)



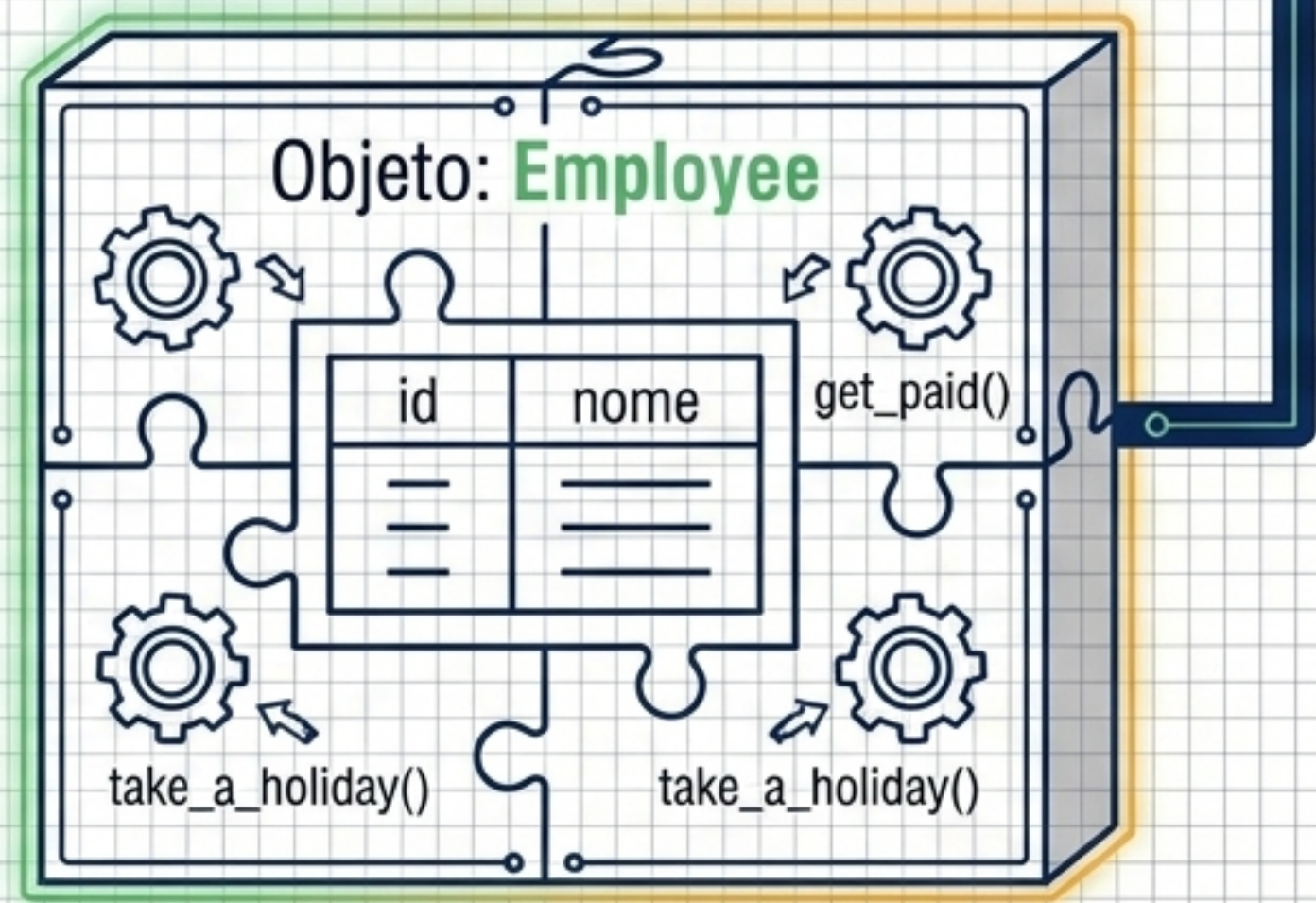
O Paradigma da Orientação a Objetos

A POO estrutura programas agrupando dados e as operações realizadas sobre esses dados em uma única entidade, acessada via objetos.

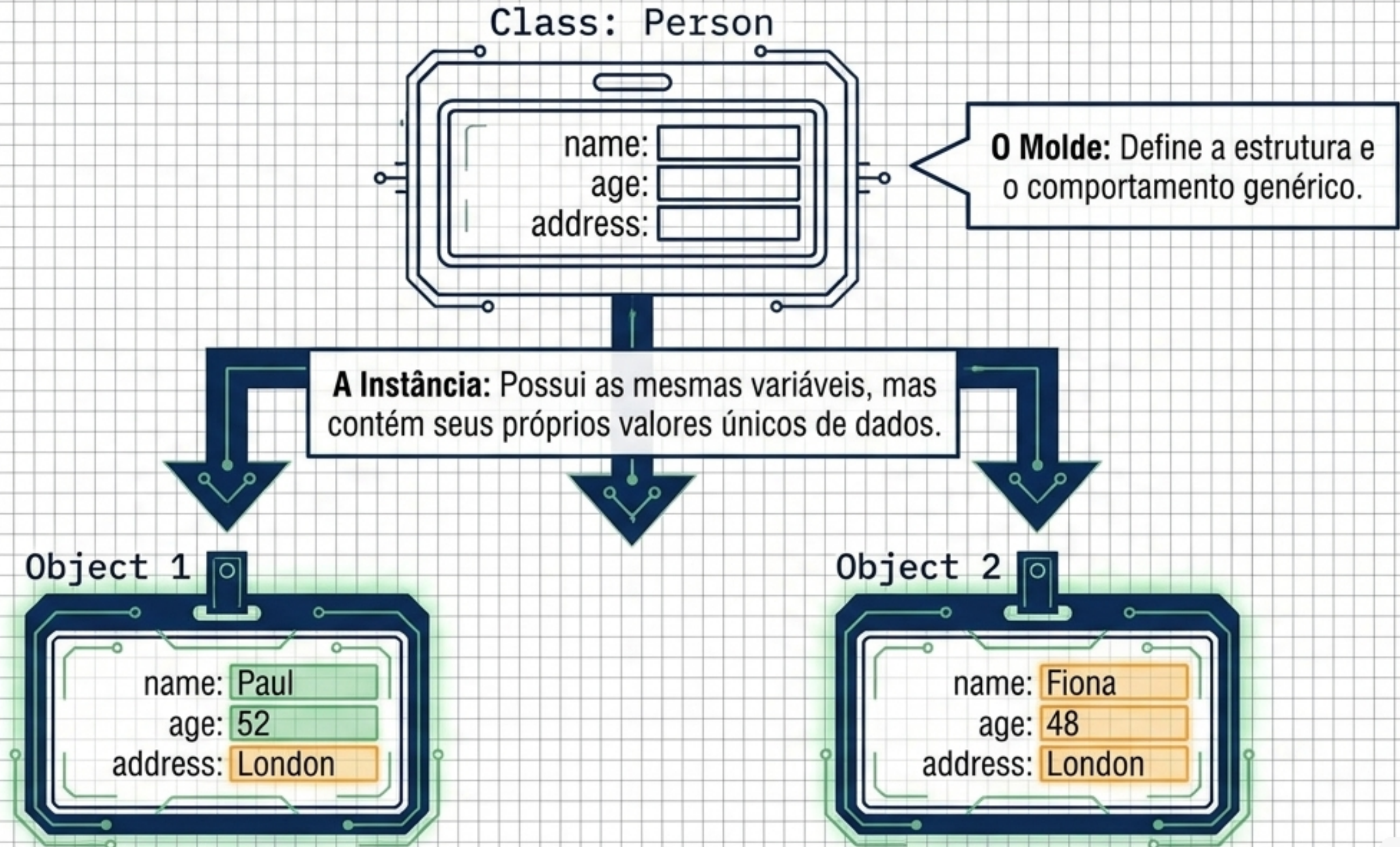
Visão Procedural



Visão Orientada a Objetos (POO)



Classes vs. Objetos

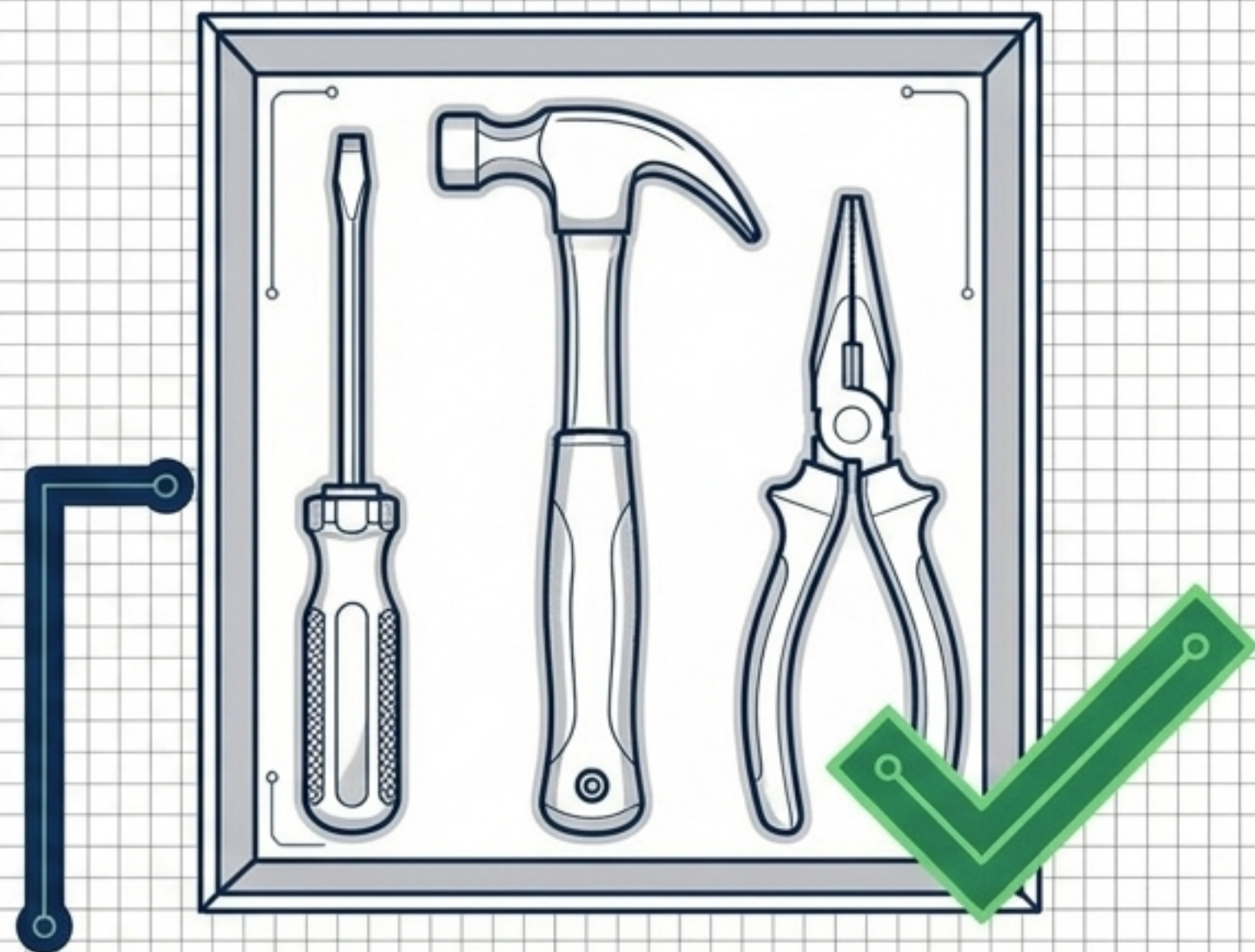


O Princípio da Responsabilidade Única

Uma classe deve ter um propósito específico e capturar apenas UMA ideia. Mesclar múltiplos conceitos reduz a reutilização e quebra o encapsulamento.

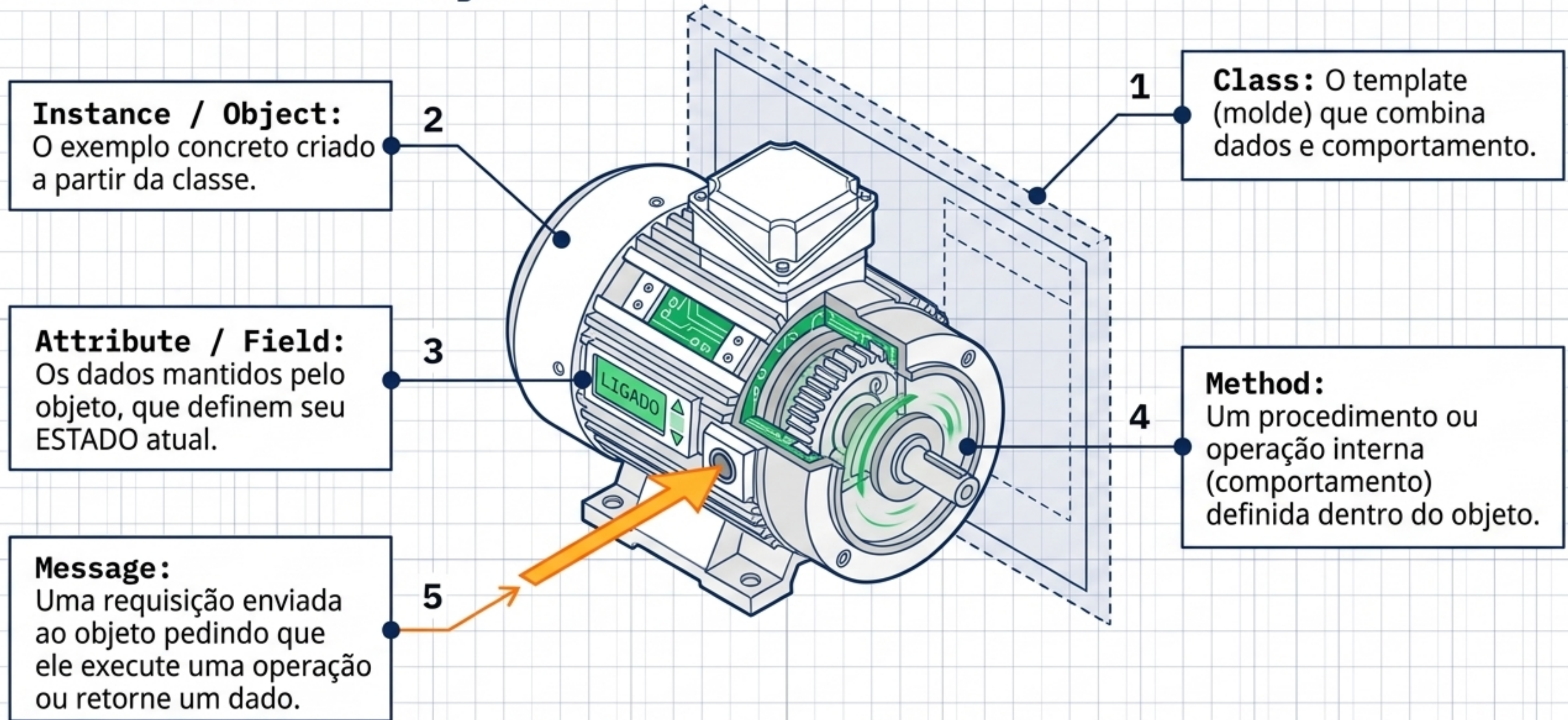


Tenta fazer de tudo e acessar dados externos.



Classes de propósito único, especializadas e coesas.

Terminologia Essencial: Anatomia de um Objeto

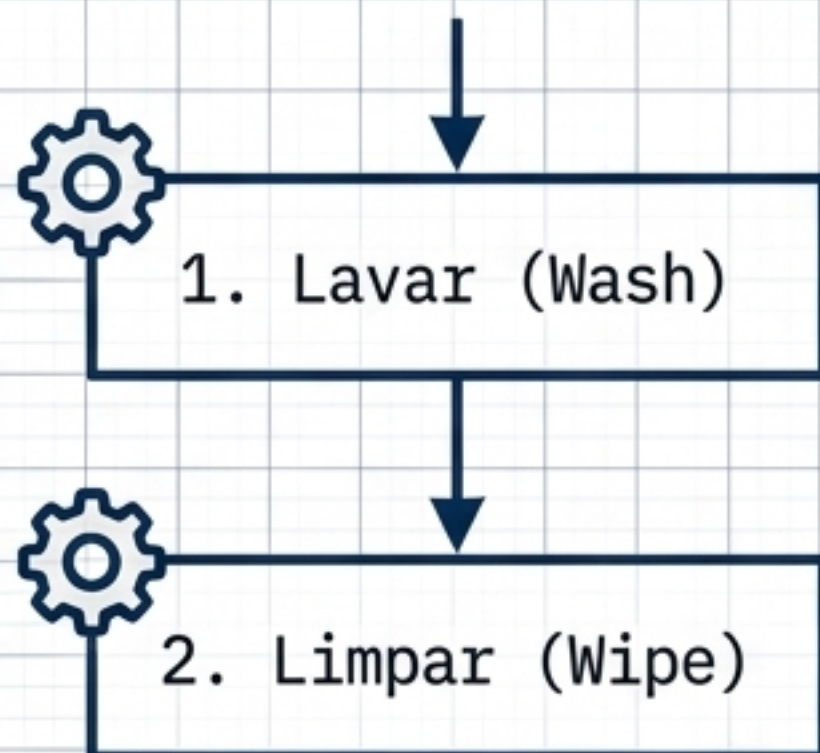


Como Construir um Sistema P00?

A grande mudança de mentalidade: Comece pelos DADOS (substantivos), não pelas funções (verbos).

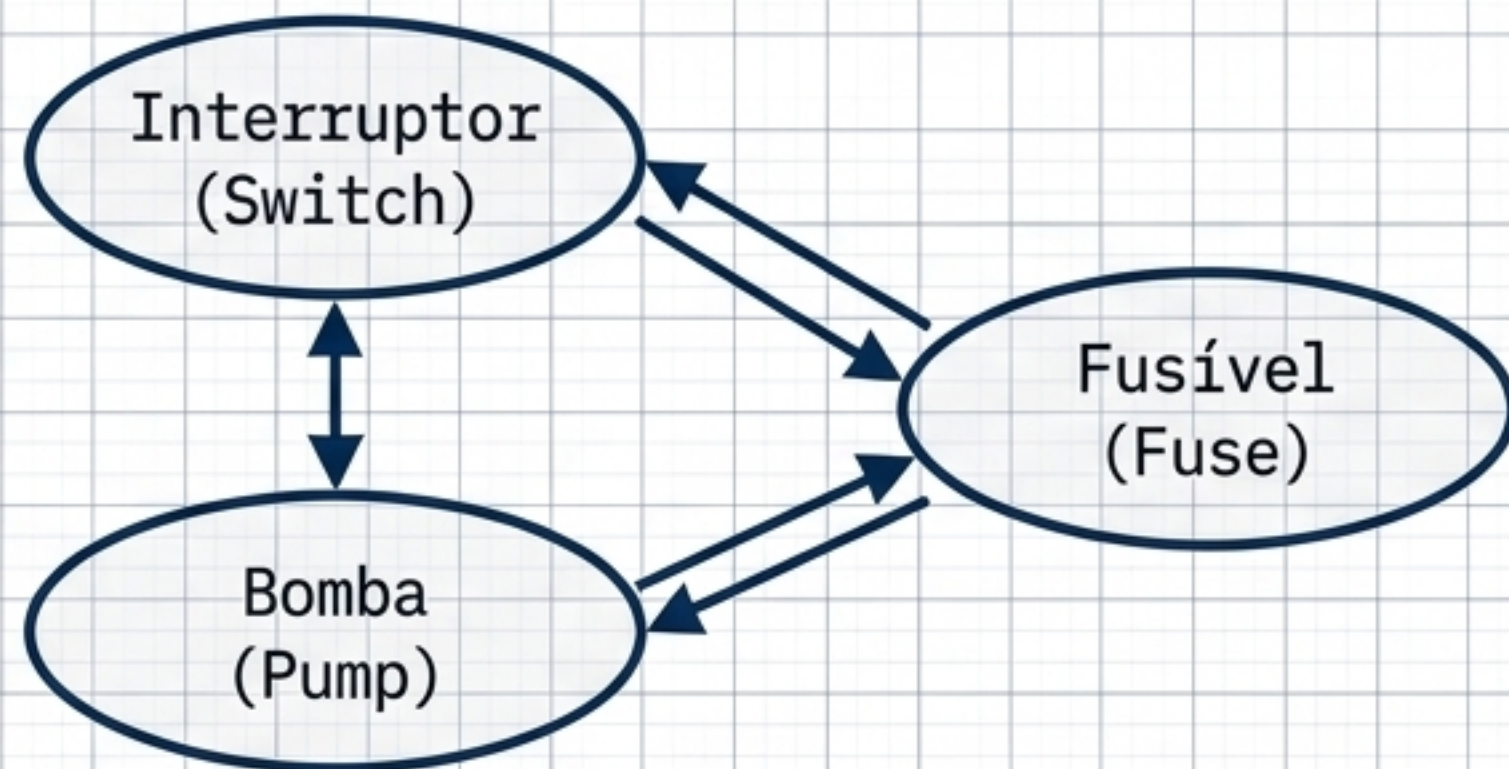
Visão Procedural

Foco nas ações (Decomposição Funcional)



Visão Orientada a Objetos

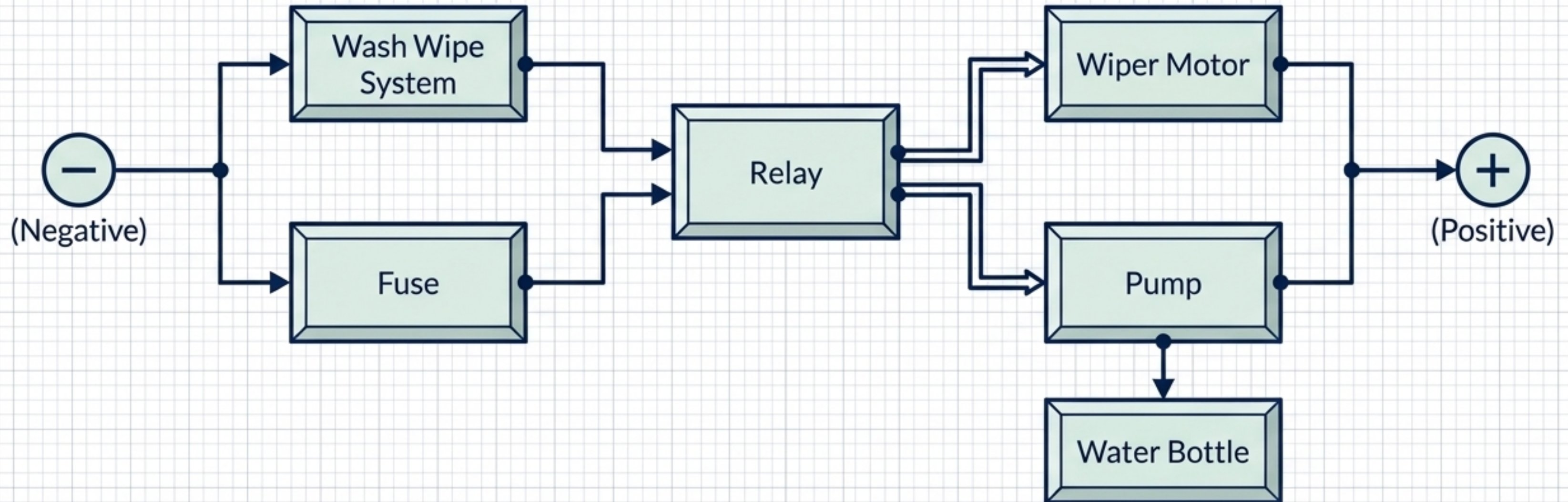
Foco nos itens de dados e relações



Em P00, os objetos **'possuem'** os dados e trocam **'mensagens'** solicitando ações sobre eles.

Estudo de Caso: Simulador de Limpador de Para-brisa

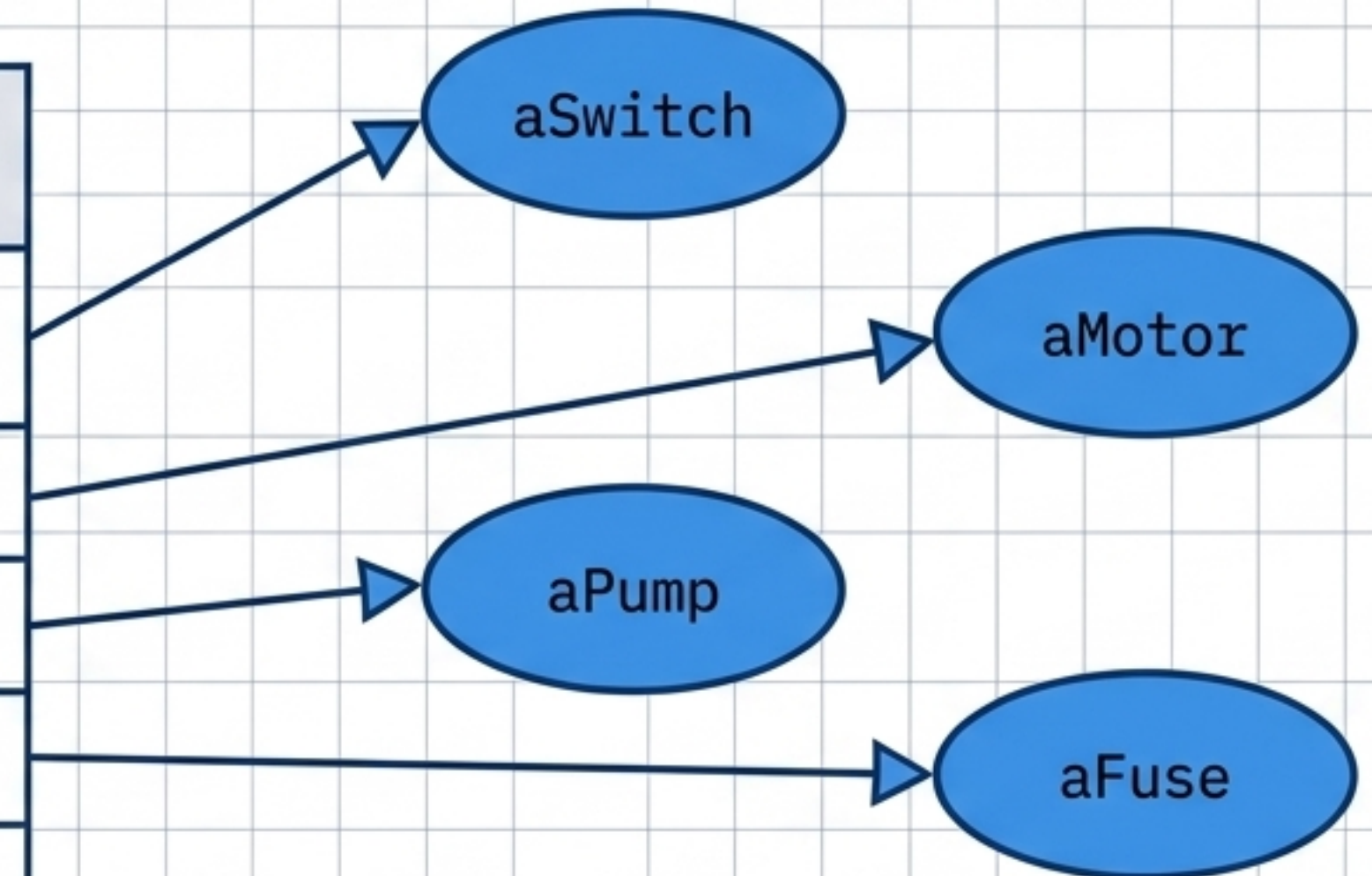
Para entender a construção de um sistema POO, vamos modelar em software um sistema de diagnóstico que simula a arquitetura mecânica de um carro real.



Passo 1: Identificando os Objetos

Olhamos para o sistema como um todo e perguntamos: o que indica o estado do sistema? Os itens de dados fundamentais tornam-se nossos objetos.

Item de Dados (Objeto)	Estado Possível (Variável de Instância)
switch_setting	off, intermittent, wipe, wash
wiper_motor	working, not
pump_state	working, not
fuse_condition	blown, not
water_bottle_level	current level
relay_status	current flowing, not



Passo 2: Definindo Serviços (Métodos)

Quais operações um usuário exige de cada objeto para alterar ou consultar seu estado?

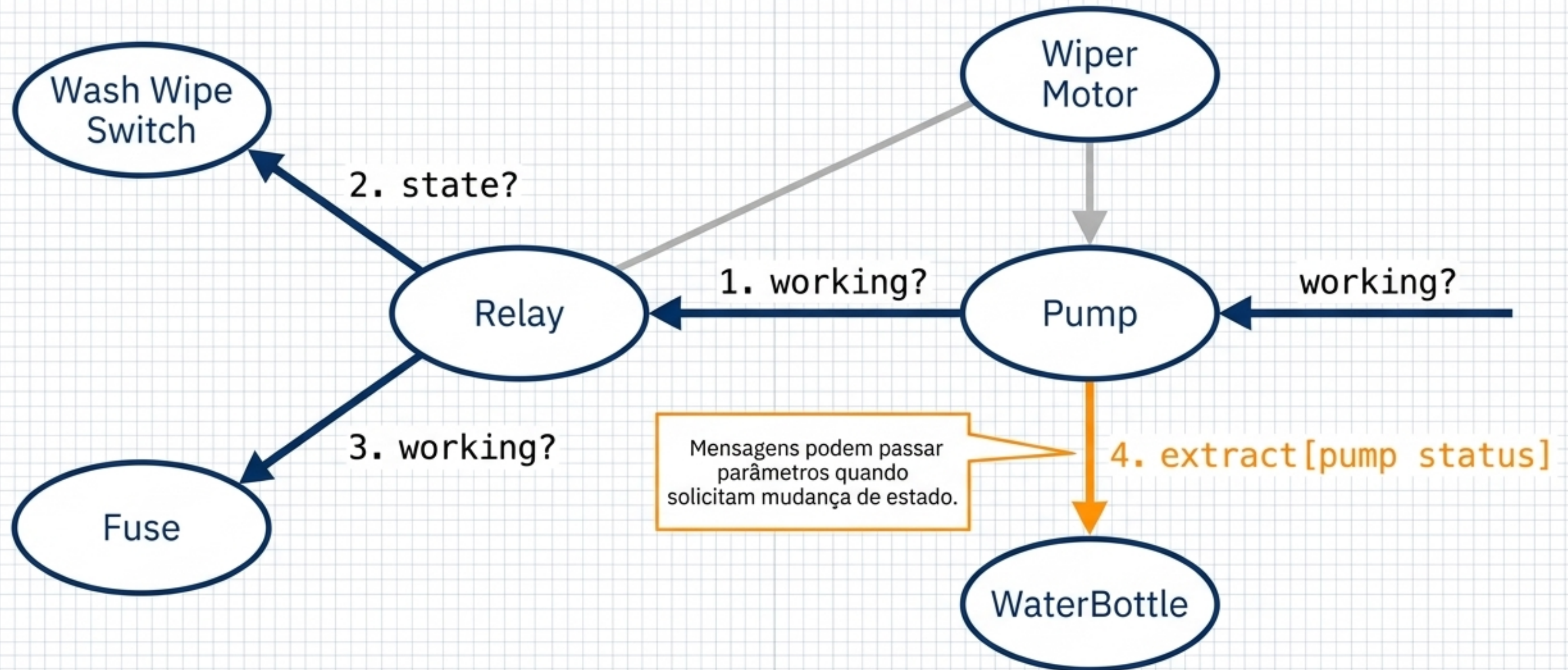
O método altera o estado interno (encapsulamento), sem expor a variável diretamente.

Objeto	Serviços (Métodos)
switch	move_up(), move_down(), state?()
water_bottle	fill(), extract(), empty()
fuse, motor, relay, pump	working?()

```
def move_up(self):  
    if self.state == 'off':  
        self.state = 'wash'  
    elif self.state == 'wash':  
        self.state = 'wipe'
```

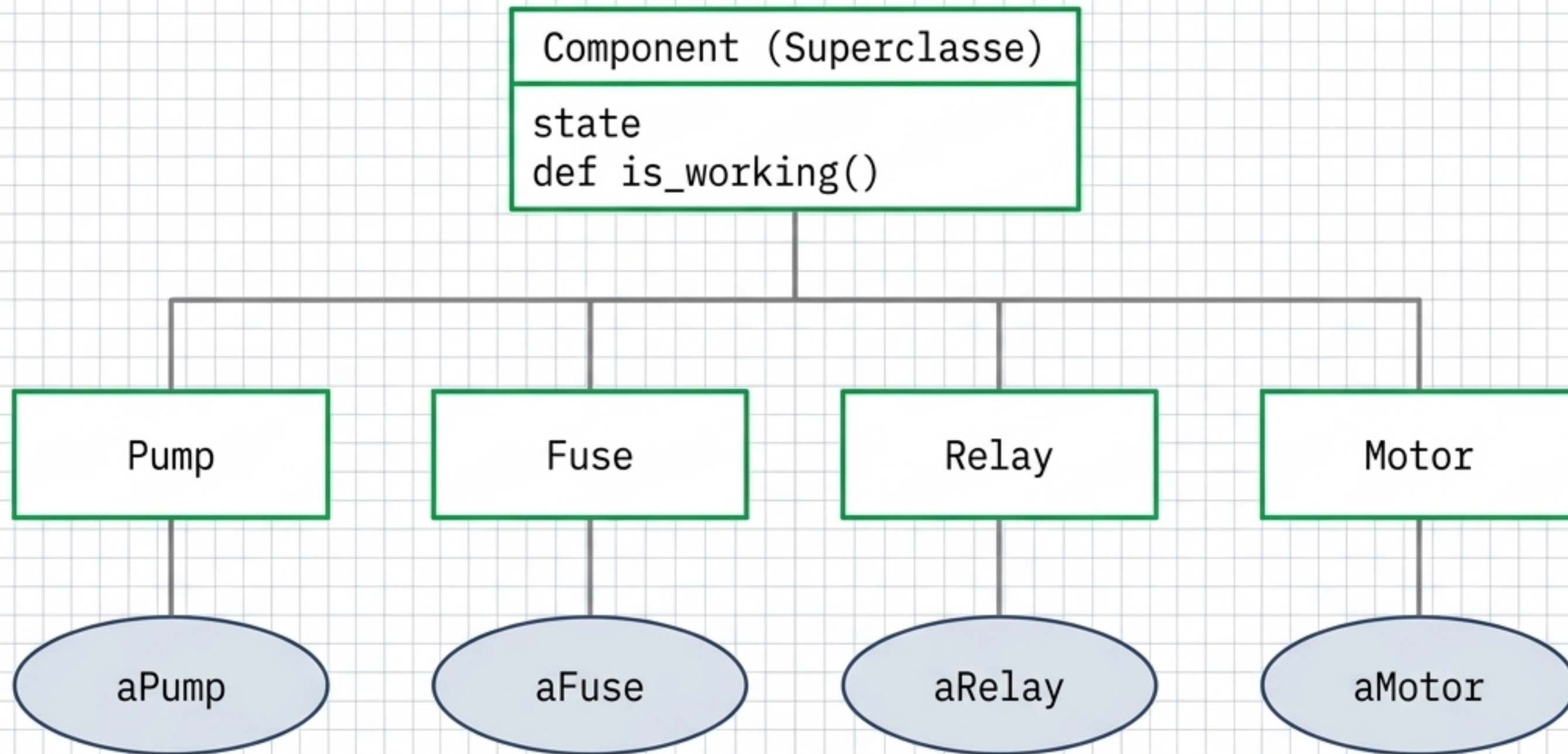

A Cadeia de Mensagens em Ação

A bomba (pump) não controla o sistema. Em vez disso, inicia-se uma rede descentralizada de objetos autônomos enviando mensagens.



Refinando a Estrutura: Herança

Fusível, Relé, Motor e Bomba compartilham o atributo 'state' e o método "working?". Otimizmos a arquitetura herdando de uma Superclasse comum.

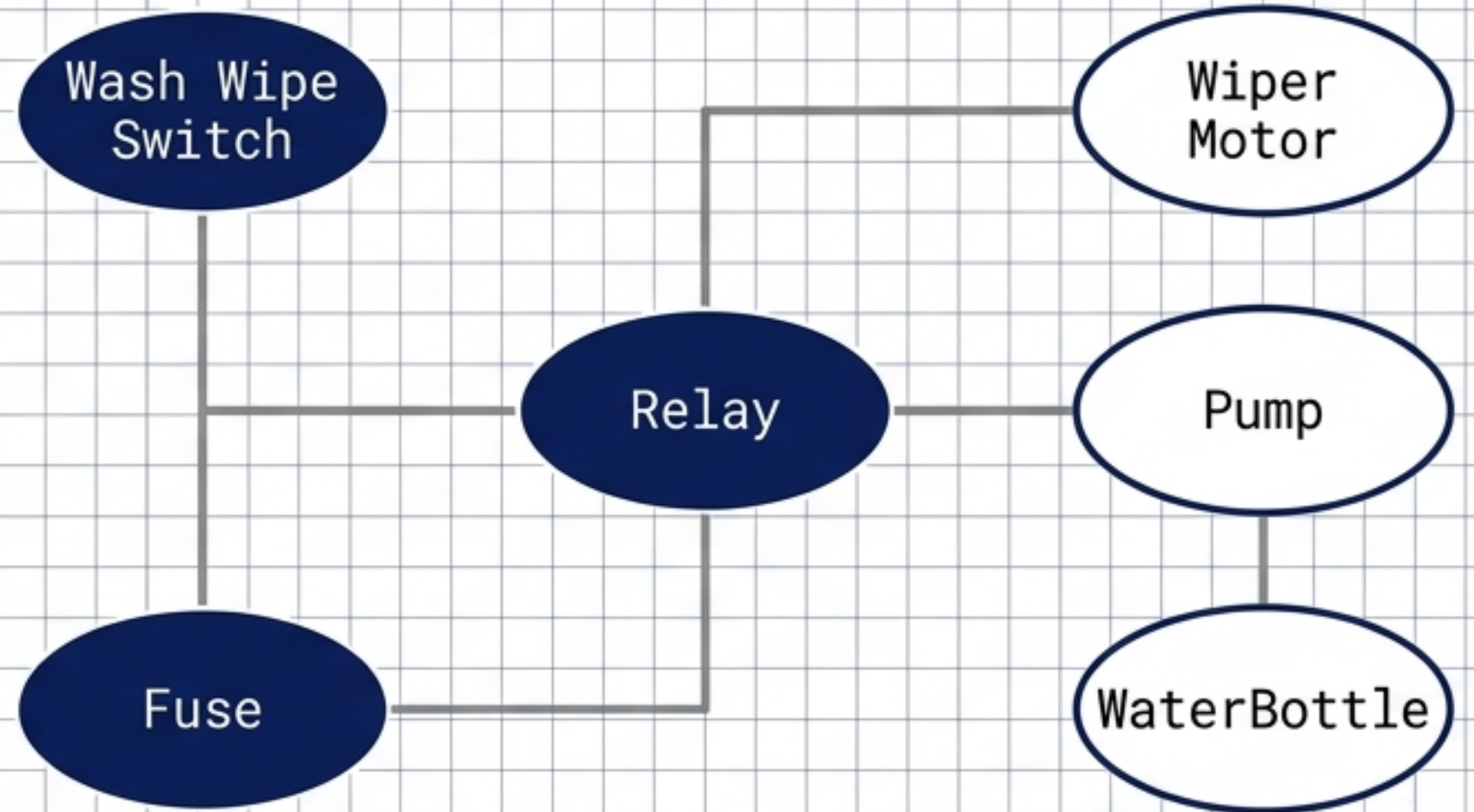


Onde está o "Main Body" do Programa?

Diferente da programação procedural, sistemas OO perdem o clássico fluxo "main" estruturado de cima para baixo. A estrutura do programa é a própria teia de objetos interconectados.

```
function Relay():  
    Beer-loadMotorstsi-Neaurts ()  
    top = top;  
    R = 6R;  
    no1 = Relay;  
    washin main()  
    process loan()  
    recipe.se  
    sectDm = 20  
    washWwFröcm  
    ...  
    wiperPump()  
    pumpmat =  
    comm.res  
    remoon  
    comm Wash Wipe Switch  
    ...  
    ...  
    comm.recareust()  
    comm.nateremin()  
end_function
```

A Estrutura Real (Referências entre Objetos)

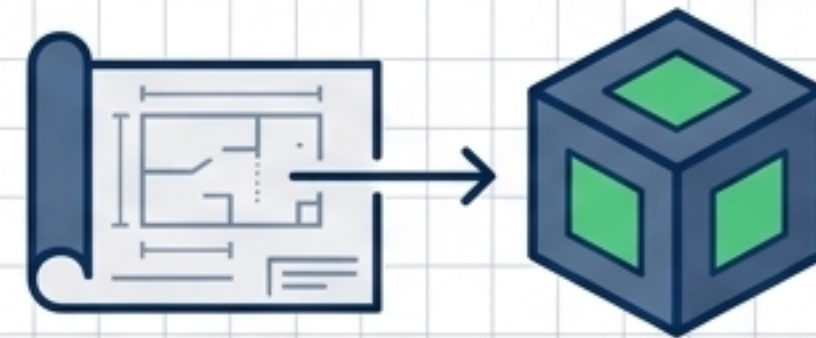


Resumo do Blueprint OO



1. Encapsulamento

Classes combinam Dados (Estado) e Operações (Comportamento) em uma única entidade coesa.



2. O Molde e o Objeto

Classes definem a estrutura genérica (o template);
Objetos (Instâncias) carregam os valores reais.



3. Foco Inicial nos Dados

O design de sistemas OO começa identificando itens de dados e seus estados, não mapeando funções.



4. Sistemas Descentralizados

O fluxo do programa não é linear; ele opera através de objetos autônomos se comunicando via Cadeias de Mensagens.